IMPLEMENTATION PLAN | SCALER × META PYTORCH HACKATHON

# FridgeEnv

## Food Waste Reduction RL Benchmark

Complete build plan for an OpenEnv-compliant environment.

Single-step episode: agent submits a full meal plan, scorer validates deterministically.

## 1 Architecture Overview

**System Diagram**

```
React Frontend (HF Space UI)
    ↓ HTTP
FastAPI Server (app.py)
    ↓ calls
FridgeEnv Core (env/)
  |- models.py      Pydantic:  Observation, Action, Reward
  |- fridge_env.py   reset(), step(), state()
  |- generator.py    Seeded fridge state factory
  |- scorer.py      Deterministic reward + grading
  |- data.py        50-item ingredient lookup
    ↓ used by
inference.py   OpenAI client LLM agent
agents/      FIFO + Random baselines
```

## 2 File Structure

```
fridge-env/
  app.py
  openenv.yaml
  inference.py
  Dockerfile
  README.md
  requirements.txt
  env/
    __init__.py
    models.py
    fridge_env.py
    generator.py
    scorer.py
    data.py
  agents/
    __init__.py
    base.py
    fifo_agent.py
    random_agent.py
  eval.py
  frontend/
    package.json
    src/
      App.jsx
      components/
        FridgeView.jsx
        MealTimeline.jsx
        ScoreCard.jsx
```

## 3 Data Layer: `env/data.py`

Hardcoded lookup table. 50 common food items. No external API calls ever.

**Item Schema**

```
INGREDIENTS = [
  {"name": "chicken_breast", "category": "protein",
   "shelf_life_days": (3, 5), "unit": "g",
   "default_qty": 500},
  {"name": "rice", "category": "carb",
   "shelf_life_days": (180, 365), "unit": "g",
   "default_qty": 1000},
  ...
]
```

Fields per item: `name`, `category` (protein | carb | vegetable | dairy | fruit | condiment), `shelf_life_days` (min, max) for jitter, `unit` (g | ml | pcs), `default_qty`.

Category distribution across 50 items: 10 protein, 10 carb, 12 vegetable, 8 dairy, 6 fruit, 4 condiment. Condiments are non-perishable and excluded from waste scoring.

**Dietary restriction tags** per item: `contains_meat`, `contains_dairy`, `contains_gluten`. Used by medium/hard tasks to enforce restrictions like "vegetarian" or "lactose-free".

## 4 Generator: `env/generator.py`

Takes a `seed` (int) and `difficulty` (str) and returns a deterministic `Observation`.

### 4.1 Difficulty Profiles

| Level | Items | Horizon | HH Size | Restrictions | Expiry Pattern |
|---|---|---|---|---|---|
| easy | 5 – 8 | 3 | 2 | none | Spread: each item expires on a different day |
| medium | 10 – 15 | 7 | 3 | 1 | Some clustering: 2 – 3 items share an expiry day |
| hard | 20 – 30 | 14 | 4 | 2 | Clusters of 4 – 6 items expiring together. Decoy long-shelf items pad the inventory. |

### 4.2 Generation Algorithm

1. Seed `random.Random(seed)`
2. Sample `n` items from `INGREDIENTS` (weighted: more perishables)
3. For each item: assign `expiry_date` = `current_date` + `randint(1, horizon + 2)`
4. Apply clustering for medium/hard: group items into expiry buckets
5. Scale quantities by `household_size`
6. For medium/hard: pick restriction(s) from [vegetarian, lactose-free, gluten-free], ensure at least 30% of items conflict with restriction
7. Return `Observation` Pydantic model

## 5 Pydantic Models: `env/models.py`

**Observation**

```
class FridgeItem(BaseModel):
  name: str
  quantity: float
  unit: str
  expiry_date: date
  category: str

class Observation(BaseModel):
  inventory: list[FridgeItem]
  current_date: date
  horizon: int
  household_size: int
  dietary_restrictions: list[str]
```

**Action**

```
class MealIngredient(BaseModel):
  name: str
  quantity: float

class Meal(BaseModel):
  day: int
  meal_name: str
  ingredients: list[MealIngredient]

class Action(BaseModel):
  meal_plan: list[Meal]
```

**Reward**

```
class Reward(BaseModel):
  score: float   # 0.0 to 1.0 (grader output)
  waste_rate: float
  nutrition_score: float
  items_used: int
  items_expired: int
  violations: list[str]
```

# 6 Core Environment: `env/fridge_env.py`

## 6.1 Interface

OpenEnv-compliant. Single-step episode (agent submits full meal plan, episode ends).

**FridgeEnv Class**

```
class FridgeEnv:
  def reset(self, task_id: str, seed: int) -> Observation:
    # Generate fridge via generator, store state
    # Return observation

  def step(self, action: Action) -> tuple[Observation, Reward, bool, dict]:
    # Validate meal plan via scorer
    # Simulate day-by-day consumption
    # Return (obs, reward, done=True, info)

  def state(self) -> dict:
    # Return current env state snapshot
```

## 6.2 Step Logic (single call, does everything)

1. Parse the Action's meal plan
2. For each day in horizon (1 to N):
   a. Collect all meals for this day
   b. For each ingredient in each meal:
      - Check: does item exist in inventory? If not → skip, log violation
      - Check: is requested quantity ≤ available? If not → clamp to available
      - Check: dietary restriction violation? If yes → log, apply penalty
      - Deduct quantity from inventory
   c. Check nutrition: does this day have ≥1 protein + ≥1 carb + ≥1 vegetable?
   d. Expire items whose `expiry_date` ≤ `current_date + day`
3. After all days processed → compute final scores via `scorer.py`
4. Return (final_obs, reward, done=True, info_dict)

# 7 Scorer: `env/scorer.py`

Pure Python. No LLM. No API calls.

## 7.1 Inputs

Original inventory (from reset), consumption log (built during step simulation), expiry events (which items expired unused), nutrition log (per-day category presence), violation log.

### 7.2 Reward Components

**Waste ratio (grader output):**

$$\text{grader_score} = \frac{\text{perishable items with} > 0\% \text{ usage before expiry}}{\text{total perishable items}} \in [0.0, 1.0]$$

**Detailed reward signal (for training, not grading):**

- Per item: $+1.0$ if fully used before expiry, proportional if partial (60% used = $+0.6$), $-1.0$ if fully expired unused
- Per day: $+0.2$ if balanced macros (protein + carb + vegetable present)
- Per violation: $-0.3$ per dietary restriction breach

**Nutrition score:**

$$\text{nutrition_score} = \frac{\text{days with balanced macros}}{\text{total days in horizon}} \in [0.0, 1.0]$$

## 8 Agents

### 8.1 Interface: `agents/base.py`

```
class BaseAgent:
  def act(self, observation:  dict) -> dict:
    raise NotImplementedError
```

Input is the Observation serialized as dict. Output is the Action serialized as dict.

### 8.2 FIFO Agent

Sort inventory by expiry date ascending. For each day, pick the 3 soonest-expiring items (one protein, one carb, one vegetable if available). Consume default portion sizes. Should score well on easy, decent on medium, poorly on hard (no restriction handling).

### 8.3 Random Agent

For each day, randomly pick $2-4$ items, random quantities ($10-50\%$ of available). Lower bound baseline.

## 9 FastAPI Server: `app.py`

Three endpoints mapping 1:1 to OpenEnv spec:

```
POST /reset   # body:  {task_id, seed} -> Observation
POST /step    # body:  Action -> {observation, reward, done, info}
GET  /state   # -> current env state
```

Server holds a single `FridgeEnv` instance. Stateful per session. React frontend and inference script both hit these endpoints. Static file serving for the built React frontend at `/`.

## 10 Inference Script: `inference.py`

Required by hackathon. Uses OpenAI Python client.

### 10.1 Flow

1. Read `API_BASE_URL`, `MODEL_NAME`, `OPENAI_API_KEY` from env vars
2. For each task in [easy, medium, hard]:
   a. For seeds $0-4$ (5 episodes per task):
      i. POST `/reset` with task_id and seed $\rightarrow$ get Observation

ii. Format observation into LLM prompt
iii. Call OpenAI API → get meal plan as JSON
iv. Parse response into Action dict
v. POST `/step` with Action → get reward + info
vi. Collect scores

b. Average scores across 5 episodes

3. Print results, save to `results.json`

### 10.2 LLM Prompt Structure

**System Message**

You are a meal planning agent. Given a fridge inventory with expiry dates, produce a meal plan that minimizes food waste. Prioritize items expiring soonest. Ensure each day has protein, carb, and vegetable. Respect dietary restrictions. Return ONLY valid JSON matching the Action schema.

**User Message Template**

```
Fridge inventory: {inventory_json}
Horizon: {horizon} days
Household size: {household_size}
Dietary restrictions: {restrictions}
Current date: {current_date}

Return JSON: {"meal_plan": [{"day": int,
  "meal_name": str, "ingredients":
  [{"name": str, "quantity": float}]}]}
```

### 10.3 Runtime Budget

5 episodes × 3 tasks = 15 LLM calls. At ∼5s per call = ∼75 seconds. Well within the 20-minute cap.

## 11 Eval Script: `eval.py`

Local baselines only (no LLM). 100 episodes per difficulty with fixed seeds 0 – 99.

**Output Format**

```
{
  "easy": {"waste_rate": 0.12, "nutrition_score": 0.85, "episodes": 100},
  "medium": {"waste_rate": 0.38, "nutrition_score": 0.52, "episodes": 100},
  "hard": {"waste_rate": 0.61, "nutrition_score": 0.28, "episodes": 100}
}
```

## 12 OpenEnv Config: `openenv.yaml`

```yaml
name: FridgeEnv
description: Food waste reduction benchmark
version: 1.0.0
tasks:
  - id: easy
    description: 5-8 items, 3-day horizon, no restrictions
    grader: scorer.compute_grader_score
  - id: medium
    description: 10-15 items, 7-day horizon, 1 restriction
    grader: scorer.compute_grader_score
  - id: hard
    description: 20-30 items, 14-day horizon, 2 restrictions
    grader: scorer.compute_grader_score
```

## 13 Docker

**Dockerfile**

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN cd frontend && npm ci && npm run build
EXPOSE 7860
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

Port 7860 is HF Spaces default. `requirements.txt`: fastapi, uvicorn, pydantic, openai.

## 14 React Frontend

Served at `/` by FastAPI. Three components:

**FridgeView.jsx:** Grid of inventory items. Card per item: name, quantity, expiry. Background color by days-until-expiry (green $>5$, yellow $3-5$, red $<3$).

**MealTimeline.jsx:** Horizontal timeline, one column per day. Meals and ingredients listed. Lines connect ingredients to fridge items.

**ScoreCard.jsx:** After step: grader score (big number), waste rate, nutrition score, violation list.

**Controls:** Task dropdown, seed input, Reset/Step buttons.

## 15 Build Order

| # | Module | Done When | Est. |
|---|---|---|---|
| 1 | `env/data.py` | 50 items with categories, shelf lives, restriction tags | 30m |
| 2 | `env/models.py` | All Pydantic models validate with sample data | 20m |
| 3 | `env/generator.py` | 3 levels produce deterministic output for same seed | 45m |
| 4 | `env/scorer.py` | Given mock consumption log, returns correct scores | 45m |
| 5 | `env/fridge_env.py` | reset → step roundtrip works with hardcoded action | 1h |
| 6 | `agents/*.py` | FIFO and Random agents produce valid actions | 30m |
| 7 | `eval.py` | 100 ep × 3 levels run, results.json written | 30m |
| 8 | `app.py` | FastAPI serves /reset, /step, /state | 30m |
| 9 | `openenv.yaml` | Passes `openenv validate` | 15m |
| 10 | `inference.py` | 15 LLM calls complete, results.json written | 45m |
| 11 | `Dockerfile` | `docker build && docker run` on 7860 | 20m |
| 12 | Frontend | React app builds, shows fridge + meals + scores | 2h |
| 13 | HF Space deploy | Space responds to /reset ping with 200 | 30m |
| 14 | README.md | Full docs | 30m |

Total: ∼**8.5 hours**. Steps 1 – 9 are the critical path for passing automated validation.

## 16 Validation Checklist

1. `openenv validate` passes
2. `docker build && docker run` starts on port 7860
3. HF Space URL returns 200
4. POST `/reset` with `{"task_id": "easy"}` returns valid Observation
5. POST `/step` with valid Action returns Reward with `done: true`
6. GET `/state` returns current state
7. `inference.py` completes in <20 min, produces `results.json`
8. All 3 tasks return grader scores in [0.0, 1.0]
9. Scores are deterministic (same seed = same score)
10. `eval.py` runs 100 episodes per difficulty without error

## 17 Risk Mitigations

| Risk | Impact | Mitigation |
|---|---|---|
| LLM returns invalid JSON | inference.py fails | try/except, 3 retries with stricter prompt. Fallback to FIFO. |
| Unsolvable state | Grader always 0 | Guarantee ≥1 protein + carb + veg expiring within horizon. |
| HF cold start timeout | Fails ping | python:3.11-slim, minimal deps, no startup compute. |
| Memory exceeded 8GB | Container killed | 50-item lookup is <10KB. No ML models. |
| openenv schema mismatch | DQ | Run validator locally before every push. Pin SDK version. |

## 18 Expected Baseline Scores

| Agent | Easy | Medium | Hard |
|---|---|---|---|
| Random | 0.2 – 0.4 | 0.1 – 0.3 | 0.05 – 0.15 |
| FIFO (greedy) | 0.7 – 0.9 | 0.4 – 0.6 | 0.2 – 0.4 |
| LLM (inference.py) | 0.8 – 1.0 | 0.5 – 0.7 | 0.3 – 0.5 |

Hard task designed so frontier models struggle below 0.5. Overlapping expiry clusters + dual dietary restrictions create combinatorial planning that greedy/naive LLM approaches cannot solve optimally.

**Deadline**

**April 8, 2026 11:59 PM**
Only team leaders can make the final submission.